Introducción a la Ciencia de Datos

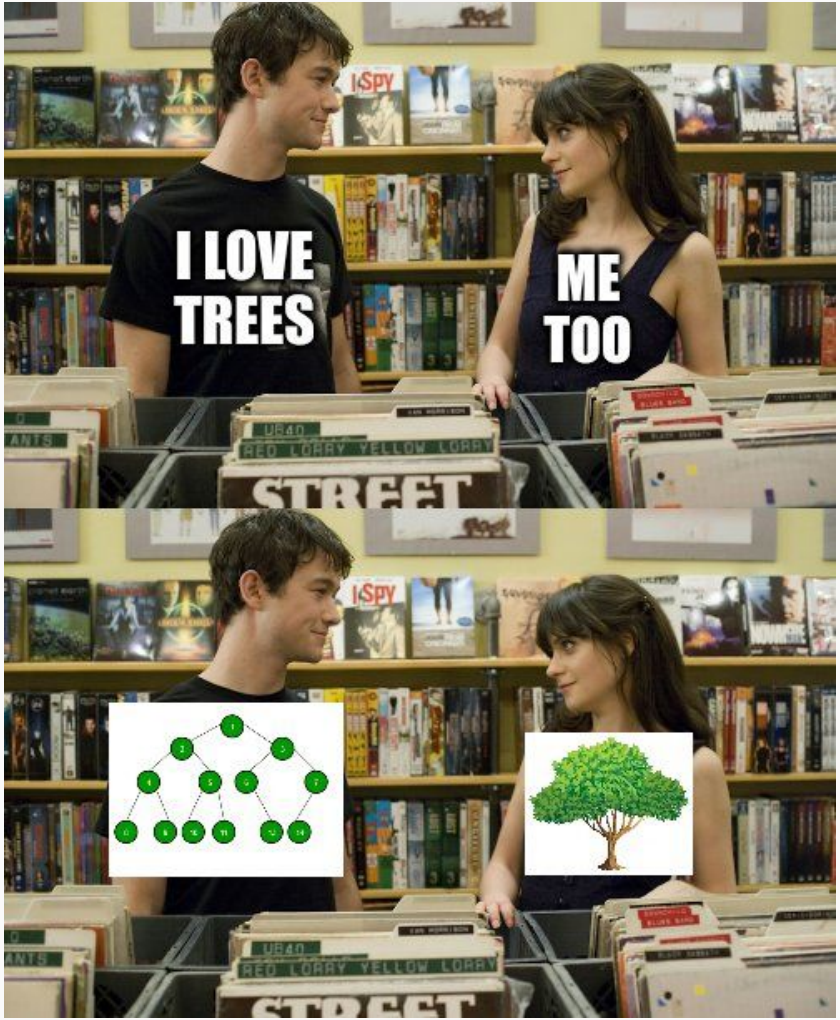
Maestría en Ciencias de la Computación

Dr. Irvin Hussein López Nava



04

Clasificación


I LOVE TREES
ME TOO
I SPY
RED LORRY YELLOW LORRY
STREET

4.2 Decision tree learning


¿Qué son los Árboles de Decisión?

Definiciones

ChatGPT

Son un tipo de modelo de aprendizaje automático que se utiliza para tareas de clasificación. Son representaciones gráficas en forma de árbol en las que se toman decisiones basadas en la información contenida en los datos.

https://chat.openai.com/

Wikipedia

Es un modelo jerárquico que utiliza un modelo de decisiones similar a un árbol y sus posibles consecuencias, incluidos los resultados de eventos casuales, los costos de los recursos y la utilidad. Es una forma de mostrar un algoritmo que solo contiene declaraciones de control condicionales.

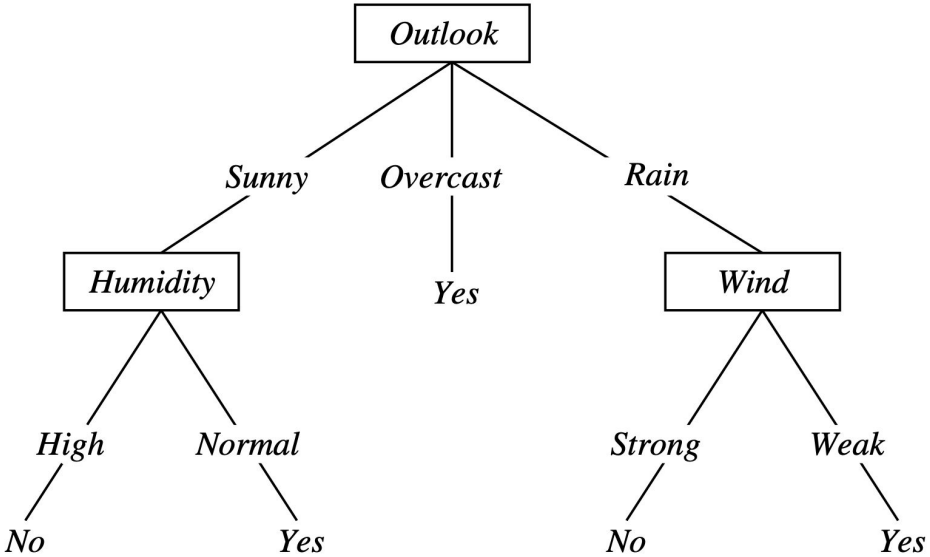
https://en.wikipedia.org/wiki/Decision_tree

Gemini

Son un tipo de algoritmo de aprendizaje supervisado que se utiliza para clasificación. Son modelos de predicción que se construyen a partir de una serie de reglas. Cada regla se divide en dos o más ramas, y cada rama representa un resultado posible.

https://gemini.google.com/

Play tennis


Outlook
Sunny
Overcast
Rain
Humidity
Yes
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes

Decision tree

- Un **árbol de decisión** representa una función que toma como entrada un vector de valores de atributos y devuelve una "decisión", i.e., un único valor de salida.
- Los valores de entrada y salida pueden ser **discretos** o **continuos**.
- Representación de un árbol de decisión:
  - Cada **nodo interno** prueba un atributo
  - Cada **rama** corresponde a un valor de atributo
  - Cada **nodo hoja** asigna una clasificación

¿Cuándo considerar los árboles de decisión?

- Instancias descriptibles por pares atributo-valor.
- La función objetivo es discreta (en ID3 y C4.5; CART también hace regresión).
- Puede ser necesaria una hipótesis disyuntiva.
- Datos de entrenamiento posiblemente ruidosos.

  Ejemplos:
  - Diagnóstico médico.
  - Análisis de riesgo de crédito y detección de fraudes.
  - Predicción de retención de empleados.
  - Recomendación de productos.

Árboles de decisión

- El algoritmo básico de aprendizaje de árboles de decisión, como el ID3 (Quinlan 1986) y su sucesor C4.5 (Quinlan 1993), utiliza una búsqueda codiciosa (*greedy*) de arriba hacia abajo a través del espacio de posibles árboles de decisión.
- Este enfoque comienza con la pregunta de qué atributo debe ser probado en la raíz del árbol.
    - Para decidir esto, cada atributo se evalúa mediante una prueba estadística para determinar qué tan bien clasifica los ejemplos de entrenamiento.

Árboles de decisión

- El mejor atributo se selecciona para ser probado en la raíz del árbol.
- Después, se crea un nodo descendiente para cada valor posible de este atributo, y los ejemplos de entrenamiento se distribuyen al nodo correspondiente según su valor.
- Este proceso se repite recursivamente en cada nodo descendiente, seleccionando el mejor atributo en cada punto del árbol, lo que constituye una búsqueda *greedy* sin retroceso para construir un árbol de decisión aceptable.

Inducción *Top-down*

Main loop:

1. **A** ← the “best” decision attribute for next node.
2. Assign **A** as decision attribute for node.
3. For each value of **A**, create new descendant of node.
4. Sort training examples to leaf nodes.
5. If training examples perfectly classified, Then STOP.
   Else iterate over new leaf nodes.

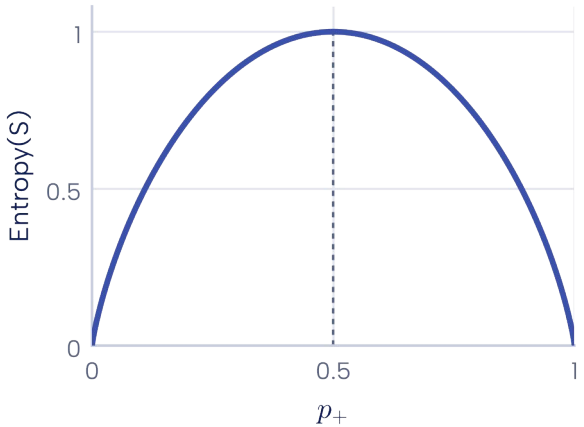
¿Cuál atributo es el mejor clasificador?

- La elección principal en el algoritmo ID3 es seleccionar qué atributo probar en cada nodo del árbol.
- Se desea elegir el atributo más útil para clasificar los ejemplos, ¿cuál es una buena medida cuantitativa del valor de un atributo?
- La propiedad estadística llamada **ganancia de información**, mide qué tan bien un atributo separa los ejemplos de entrenamiento según su clasificación objetivo.
- ID3 utiliza esta medida para seleccionar entre los atributos candidatos en cada paso mientras construye el árbol.

Entropía

- **S** es una muestra de ejemplos de entrenamiento.
- p_+ es la proporción de ejemplos positivos en *S*.
- p_- es la proporción de ejemplos negativos en *S*.
- La **entropía** mide la impureza de *S*:

$$\text{Entropy}(S) \equiv -\ p_+ \log_2 p_+ \ -\ p_- \log_2 p_-$$



Aplicando la Entropía

- *Entropy(S)* = número esperado de bits necesarios para codificar la clase *(S+ o S–)* de un ejemplo de S extraído al azar.
- Teoría de la información: el código de longitud óptima asigna bits $-log_2 p$ al mensaje que tiene probabilidad *p*.

$$Entropy(S) \equiv \sum_{i=1}^{c} -p_i \ log_2 \ p_i$$

Aplicando la Entropía

- Para ilustrar esto, suponga que *S* es una colección de 14 ejemplos de algún concepto booleano, incluidos 9 ejemplos positivos y 5 negativos (adoptando la notación [9+, 5−] para resumir dicha muestra de datos).
- Entonces, la entropía de *S* relativa a esta clasificación booleana es

$$Entropy(S) \equiv \sum_{i=1}^{c} -p_i \log_2 p_i$$

$$Entropy([9+, 5-]) = -(9/14) \log_2(9/14) - (5/14) \log_2(5/14) = 0.940$$

Aplicando la Entropía

- La entropía es 0 si todos los ejemplos de *S* son de la misma clase. Si todos son positivos ($p_+ = 1$, $p_- = 0$):

$$\text{Entropy}(S) = -1 \cdot \log_2 1 \; - \; 0 \cdot \log_2 0 \; = \; 0$$

  - Por convención, $0 \cdot \log_2 0 = 0$: es el límite de $p \log_2 p$ cuando $p \to 0$.
- Con dos clases vale 1 cuando hay tantos positivos como negativos, y está entre 0 y 1 en los demás casos.
- Con *c* clases, el máximo es $\log_2 c$, cuando todas las clases son igual de frecuentes.

Ganancia de información

- *Gain(S, A)* = reducción esperada de entropía debido a la clasificación en A causada por la partición de los ejemplos según este atributo.

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

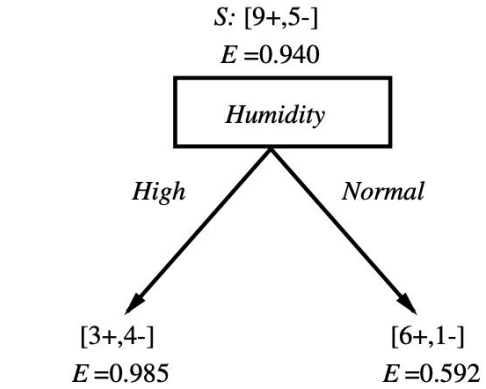
  donde *Values(A)* es el conjunto de todos los valores posibles para el atributo A, y S, es el subconjunto de S para el que el atributo A tiene valor *v*.
- Note que el primer término de la ecuación es la entropía de la colección original *S*, y el segundo término es el valor esperado de la entropía después de que *S* se particione utilizando el atributo *A*.

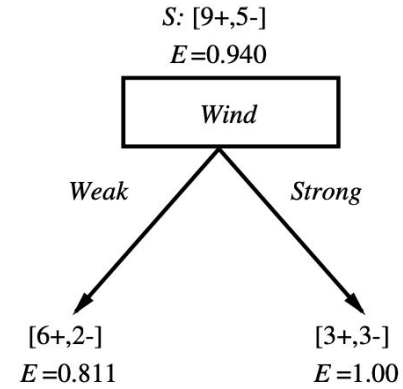
Attributes

day	outlook	temp	humidity	wind	play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Seleccionar el atributo siguiente

- ¿Cuál es el mejor atributo para clasificar?





Gain (S, Humidity)

= .940 - (7/14).985 - (7/14).592
= .151

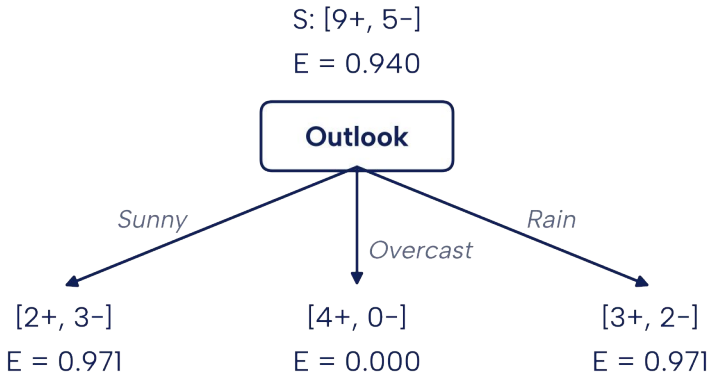
Gain (S, Wind)

= .940 - (8/14).811 - (6/14)1.0
= .048

Outlook?

Temperature?

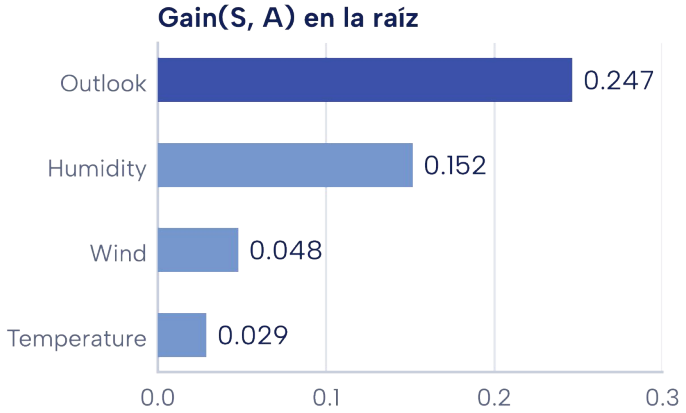
El mejor atributo para la raíz



Gain (S, Outlook)

= 0.940 - (5/14) 0.971 - (4/14) 0.0 - (5/14) 0.971

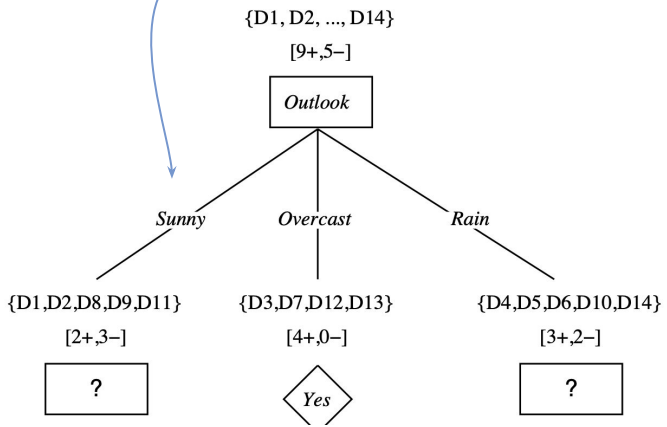
= 0.247



Outlook tiene la mayor ganancia: va en la raíz.

day	outlook	temp	humidity	wind	play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

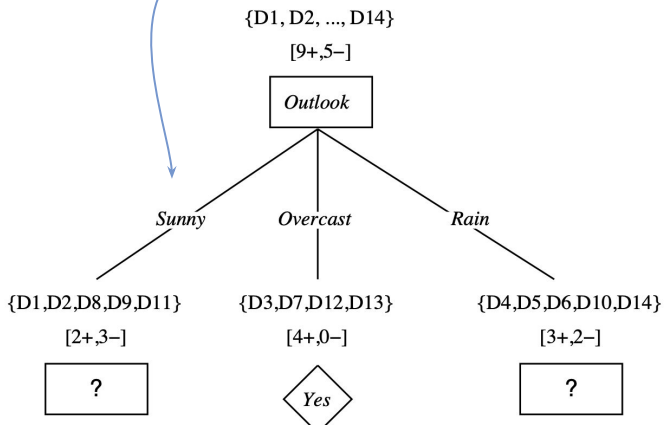
¿y el que sigue? La rama *Sunny*



S_{Sunny} = {D1, D2, D8, D9, D11}

- $Gain(S_{Sunny}, Temperature)$ =
- $Gain(S_{Sunny}, Humidity)$ =
- $Gain(S_{Sunny}, Wind)$ =

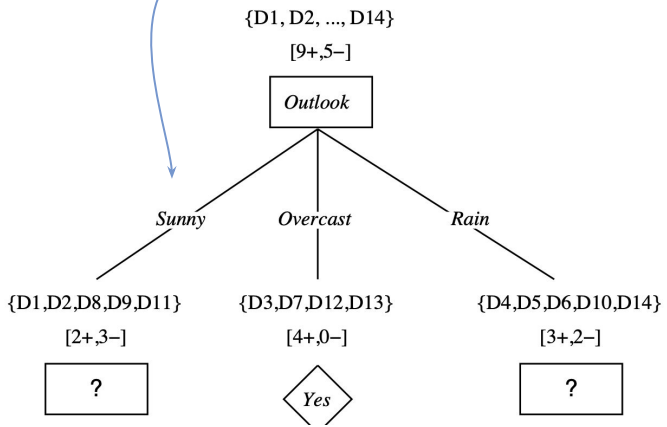
¿y el que sigue? La rama *Sunny*



S_{Sunny} = {D1, D2, D8, D9, D11}

- Gain(S_{Sunny}, Temperature) = 0.971 − (2/5) 0.0 − (2/5) 1.0 − (1/5) 0.0 = 0.571
- Gain(S_{Sunny}, Humidity) =
- Gain(S_{Sunny}, Wind) =

¿y el que sigue? La rama *Sunny*



S_{Sunny} = {D1, D2, D8, D9, D11}

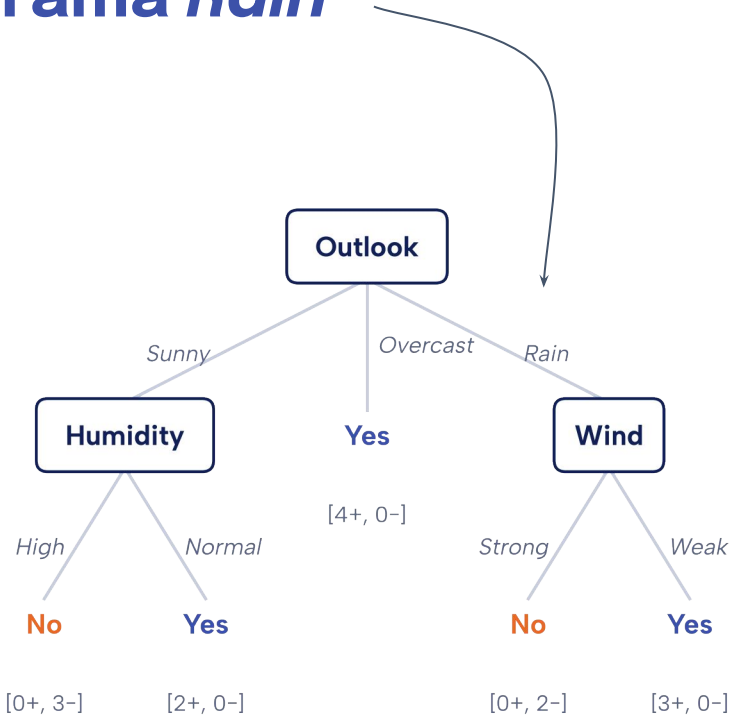
- Gain(S_{Sunny}, Temperature) = 0.971 – (2/5) 0.0 – (2/5) 1.0 – (1/5) 0.0 = 0.571
- Gain(S_{Sunny}, Humidity) = 0.971 – (3/5) 0.0 – (2/5) 0.0 = **0.971**
- Gain(S_{Sunny}, Wind) = 0.971 – (2/5) 1.0 – (3/5) 0.918 = 0.020

¿y el que sigue? La rama *Rain*

S_{Rain} = {D4, D5, D6, D10, D14}: [3+, 2−], E = 0.971

- *Gain*(S_{Rain}, *Temperature*)
  = 0.971 − (3/5) 0.918 − (2/5) 1.0 = 0.020
- *Gain*(S_{Rain}, *Humidity*)
  = 0.971 − (2/5) 1.0 − (3/5) 0.918 = 0.020
- *Gain*(S_{Rain}, *Wind*)
  = 0.971 − (3/5) 0.0 − (2/5) 0.0 = **0.971**

Wind separa la rama por completo: Weak [3+, 0−] y Strong [0+, 2−].

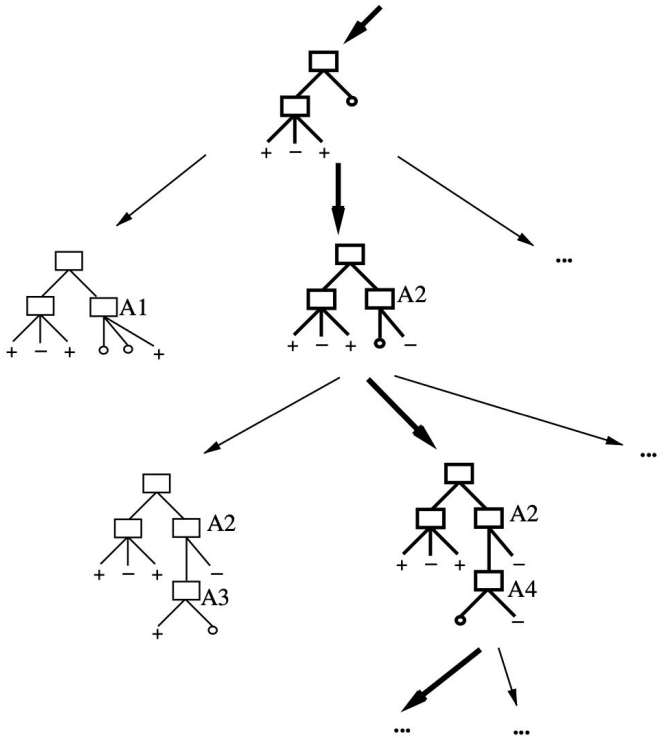


Algoritmo ID3



ID3 (Examples, Target_Attribute, Attributes)
Create a root node for the tree
If all examples are positive, **Return** the single-node tree Root, with label = +.
If all examples are negative, **Return** the single-node tree Root, with label = -.
If number of predicting attributes is empty, **then** Return the single node tree Root,
with label = most common value of the target attribute in the examples.
Otherwise Begin
A ← The Attribute that best classifies examples.
Decision Tree attribute for Root = A.
For each possible value, v_i, of A,
Add a new tree branch below Root, corresponding to the test A = v_i.
Let Examples(v_i) be the subset of examples that have the value v_i for A
If Examples(v_i) is empty
Then below this new branch add a leaf node with label = most common target value in the examples
Else below this new branch add the subtree ID3 (Examples(v_i), Target_Attribute, Attributes – {A})
End
Return Root

Búsqueda de espacio de hipótesis por ID3



Búsqueda de espacio de hipótesis por ID3

- El espacio de hipótesis es **completo**: contiene todas las funciones discretas finitas de los atributos.
  - La función objetivo seguramente está ahí.
- Mantiene **una sola hipótesis**.
  - No sabe cuántos árboles son consistentes con los datos, ni qué ejemplo preguntar para distinguirlos, como en «20 preguntas».
- **No retrocede**: lo que elige en un nodo no se reconsidera.
  - Puede quedarse en un óptimo local.
- Elige con **estadísticas** de todos los ejemplos del nodo.
  - Tolera ejemplos con ruido, si no se le deja ajustarlos todos.
- **Sesgo inductivo**: prefiere árboles cortos.

Sesgo inductivo en ID3

Notar que H es el conjunto potencia de instancias X
→¿Imparcial?

No precisamente...

- Preferencia por árboles pequeños y por aquellos con atributos de alta ganancia de información cerca de la raíz.
- El sesgo es una preferencia por alguna hipótesis, en lugar de una restricción del espacio de hipótesis H.
- *Occam's razor*: preferir la hipótesis más corta que se ajuste a los datos.

¿Por qué preferir hipótesis pequeñas?

Argumento a favor:

- Hay menos hipótesis cortas que largas
- Es improbable que una hipótesis corta que se ajuste a los datos sea una coincidencia
    - Una hipótesis larga que se ajuste a los datos podría ser una coincidencia.

Argumento opuesto:

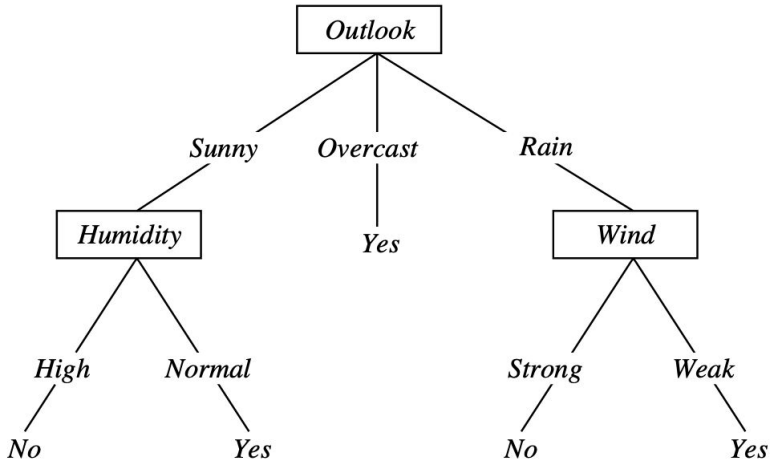
- Hay muchas formas de definir conjuntos pequeños de hipótesis
- ¿Qué tienen de especial los conjuntos pequeños basados en el tamaño de la hipótesis?

Sobreajuste en árboles de decisión

Considere agregar un ejemplo de entrenamiento **ruidoso** (n.º 15):

Sunny, Hot, Normal, Strong, Play = No

¿Qué efecto tiene sobre el árbol anterior?



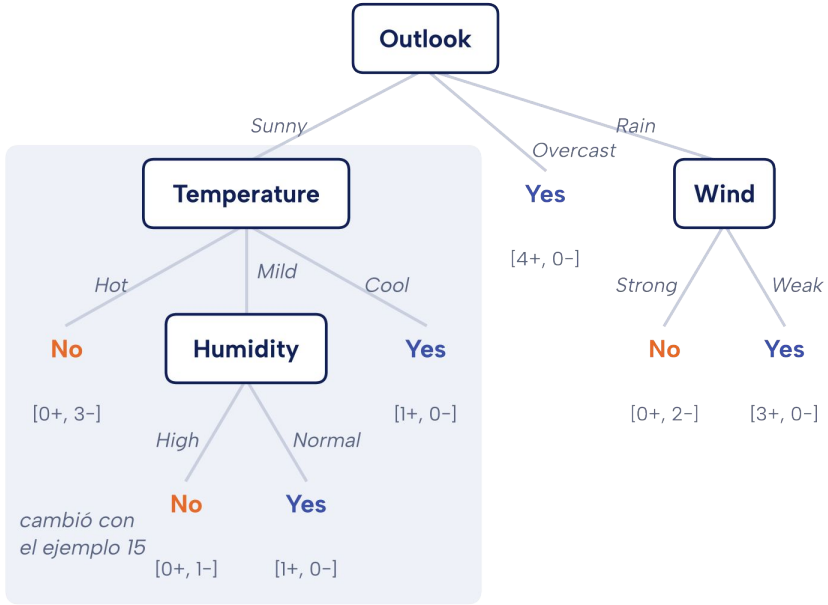
¿Qué efecto tiene el ejemplo 15?

ID3 recalcula todo. La raíz sigue siendo Outlook (0.280); la rama Sunny ahora tiene [2+, 4−]:

- *Gain*(S_{Sunny}, *Temperature*) = **0.585**
- *Gain*(S_{Sunny}, *Humidity*) = 0.459
- *Gain*(S_{Sunny}, *Wind*) = 0.000

Temperature desplaza a Humidity y Mild necesita otra prueba: **7 hojas en lugar de 5**.

Cambia lo que predice para días que nunca vio: Sunny, Cool, High era No y ahora es Yes.



Sobreajuste

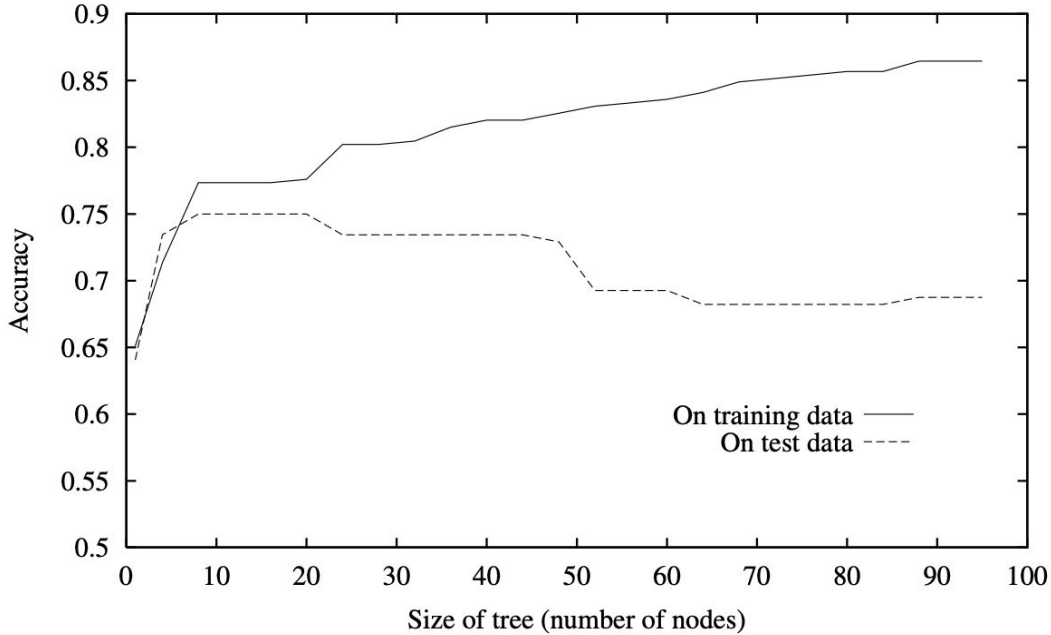
Considere el error de la hipótesis *h* sobre

- datos de entrenamiento: $error_{train}(h)$
- distribución completa D de datos: $error_{D}(h)$

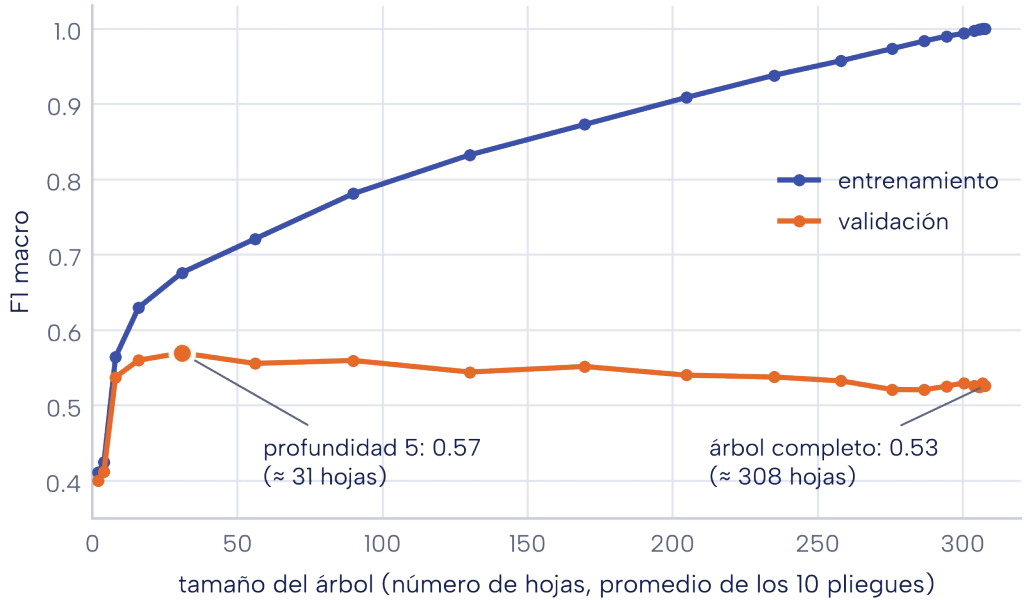
La hipótesis $h \in H$ sobreajusta los datos de entrenamiento si existe una hipótesis alternativa $h' \in H$ tal que

$$error_{train}(h) < error_{train}(h')$$
y
$$error_{D}(h) > error_{D}(h')$$

Sobreajuste en árbol de decisiones



Sobreajuste con nuestros datos



Vino de 3 clases (sesión 12), 10 pliegues, duplicados en el mismo pliegue.

Cada punto es un árbol con `max_depth` de 1 a 20; el último, sin límite.

Entrenamiento llega a 1.00. Validación: máximo de **0.57** con profundidad 5; el árbol completo baja a 0.53.

¿Cómo evitar el sobreajuste?

- Dejar de crecer el árbol cuando la división de datos no es estadísticamente significativa.
- Crecer el árbol completo y luego realizar una poda posterior.

¿Cómo seleccionar el "mejor" árbol?:

- Usar todos los datos de entrenamiento y una prueba estadística (χ^2).
- Medir el rendimiento en un conjunto de datos de validación independiente.
- MDL: minimizar

$$size(tree) + size(misclassifications(tree))$$

Podado de error-reducido

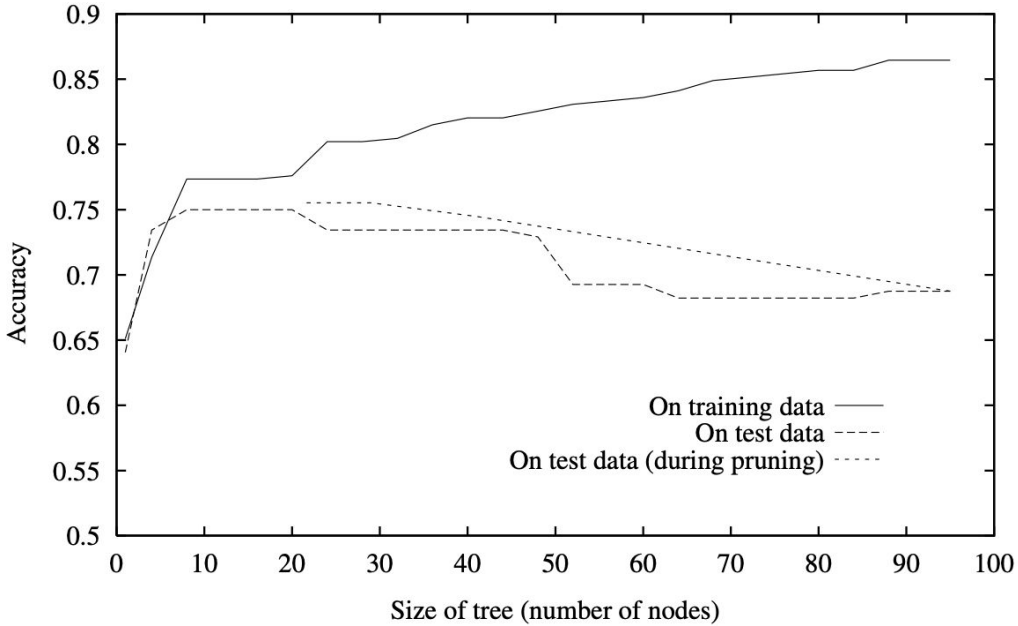
Dividir los datos en entrenamiento y validación. La prueba se aparta desde el inicio y se usa una sola vez.

Do until further pruning is harmful:

1. Evaluate the impact on validation set of pruning each possible node (plus those below it).
2. Greedily remove the one that most improves validation set accuracy.

⇒ Produce la versión más pequeña del subárbol más preciso.
¿Qué pasa si los datos son limitados?

Efectos del podado

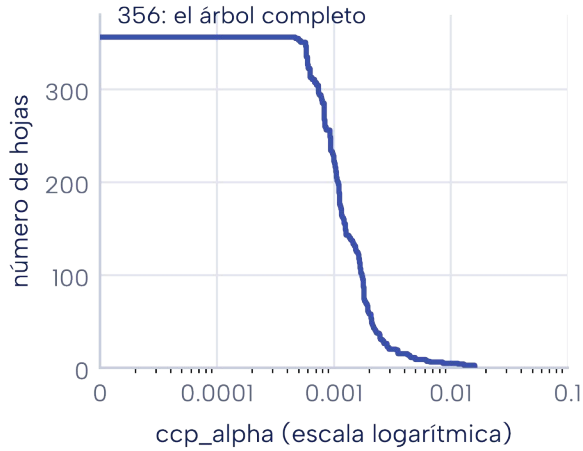


Podar en scikit-learn

- **Prepoda**: no dejar crecer el árbol.
  - `max_depth`: profundidad máxima.
  - `min_samples_leaf`: ejemplos mínimos por hoja.
- **Pospoda por costo-complejidad** (CART): crecer el árbol completo y podar las ramas cuyo aporte no compensa sus hojas:

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

  - $R(T)$: impureza de las hojas; $|T|$: número de hojas; α = `ccp_alpha`.



Ni la profundidad ni α se eligen con el entrenamiento: se eligen con validación.

¿Cuánto podar? Se decide con validación



	hojas	F1 macro, validación	F1 macro, prueba
árbol completo	274	0.551	0.484
árbol elegido	**55**	**0.584**	**0.551**
regresión logística	—	0.562	0.573

La validación del árbol elegido (0.584) es optimista: es la mejor de 40 combinaciones. Por eso la prueba se usa una sola vez.

Con pocos datos, la validación cruzada sustituye al conjunto de validación fijo.

Reglas post - podado

1. Convertir el árbol en un conjunto equivalente de reglas.
2. Podar cada regla independientemente de las demás.
3. Ordenar las reglas finales en la secuencia deseada para su uso.

Quizá sea el enfoque más utilizado (e.g., C4.5 de Ross Quinlan).



https://www.rulequest.com/Personal/

Convertir árbol en reglas de clasificación

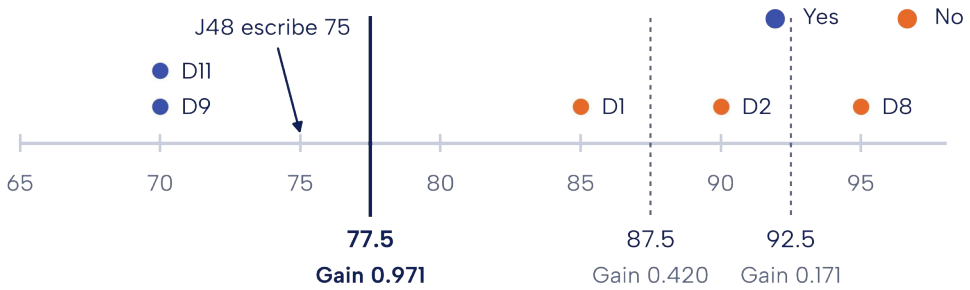
Definición: una regla de decisión tiene la forma **SI** ‹condiciones› **ENTONCES** ‹clase›. Cada camino de la raíz a una hoja es una regla:

SI Outlook = Sunny ∧ Humidity = High **ENTONCES No**
SI Outlook = Sunny ∧ Humidity = Normal **ENTONCES Yes**
SI Outlook = Overcast **ENTONCES Yes**
SI Outlook = Rain ∧ Wind = Strong **ENTONCES No**
SI Outlook = Rain ∧ Wind = Weak **ENTONCES Yes**

- Podar una regla es quitarle una precondición si eso no empeora su precisión estimada.
  - Sin «Outlook = Sunny», la primera queda «SI Humidity = High ENTONCES No», que acierta 4 de 7 días en lugar de 3 de 3: no se poda.
- Cada regla se poda por separado: una prueba puede salir de una regla y quedarse en otra.

Atributos continuos

- **Discretizar antes de aprender**: los cortes Humidity > 82 y Temperature > 76 y > 69 reproducen la versión nominal.
- **Elegir el umbral al aprender**, como C4.5: probar los cortes entre valores consecutivos y quedarse con el de mayor ganancia. En la rama Sunny:



J48 con weather.numeric

```
outlook = sunny
|   humidity <= 75: yes (2.0)
|   humidity > 75: no (3.0)
outlook = overcast: yes (4.0)
outlook = rainy
|   windy = TRUE: no (2.0)
|   windy = FALSE: yes (3.0)
```

J48 escribe 75: el mayor valor de todo el entrenamiento que no pasa de 77.5. scikit-learn escribe el punto medio.

Atributos continuos

Transformar a atributos discretos que representen el dominio continuo.

- (Humidity > 82) = High, Normal

- Temperature > 76 = Hot
- Temperature > 69 = Mild
- Temperature <70 = Cool

Atributos con muchos valores

Problema:

- Si un atributo tiene muchos valores, *Gain* lo seleccionará.
- Una solución: utilizar *GainRatio* en su lugar:

$$GainRatio(S, A) \equiv \frac{Gain(S,A)}{SplitInformation(S,A)}$$

$$SplitInformation(S, A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} log_2 \frac{|S_i|}{|S|}$$

- donde S_i es el subconjunto de S para el que A tiene el valor v_i.

¿Y el atributo Day?

Atributo	Gain	SplitInformation	GainRatio
Day	**0.940**	**3.807**	**0.247**
Outlook	0.247	1.577	0.156
Humidity	0.152	1.000	0.152
Wind	0.048	0.985	0.049
Temperature	0.029	1.557	0.019

- Day separa los 14 días en 14 ramas puras: la ganancia máxima posible.
- GainRatio lo castiga, pero aun así gana: 0.247 contra 0.156 de Outlook.
- Con Day en la raíz, el árbol no puede clasificar ningún día nuevo: el día 15 no tiene rama.

Lo que de verdad lo detiene:

- Day no es un atributo, es un identificador (sesión 2): se quita antes de entrenar.
- En J48, una división necesita al menos dos ramas con 2 ejemplos o más (`minNumObj` = 2).

Atributos con costos

- ¿Cómo aprender un árbol cuando medir cada atributo tiene un costo, como un análisis clínico?
- Un enfoque: sustituir *Gain* por

$$\frac{Gain^2(S,A)}{Cost(A)}$$

$$\frac{2^{Gain(S,A)}-1}{(Cost(A)+1)^w}$$

- donde w ∈ [0,1] determina la importancia del costo frente a la ganancia.

Valores de atributo desconocidos

¿Qué pasa si en algunos ejemplos faltan valores de A?

Usar el ejemplo de todos modos y bajarlo por el árbol:

- Si el nodo n prueba A, asignar el valor más común de A entre los ejemplos que llegan a *n*.
- Asignar el valor más común de A entre los ejemplos de la misma clase.
- Asignar una probabilidad p_i a cada valor posible v_i de A.
    - Mandar una fracción p_i del ejemplo a cada rama (así lo hace C4.5).

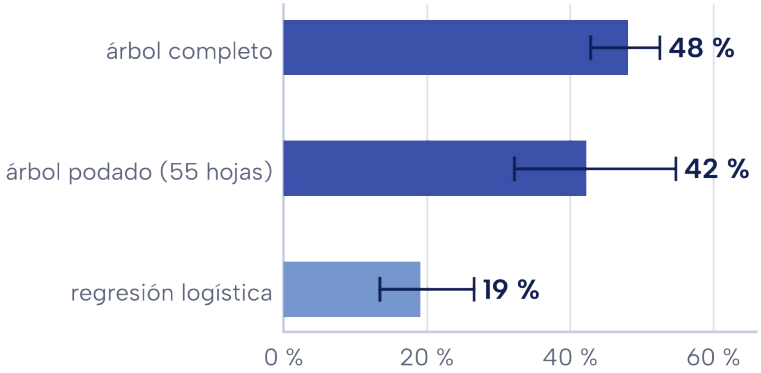
Los ejemplos nuevos se clasifican de la misma manera.

ID3, C4.5 y CART

	ID3	C4.5 (J48 en Weka)	CART (scikit-learn)
Divisiones	una rama por valor	por valor; binaria en continuos	siempre binaria
Criterio	Gain	GainRatio	Gini (o entropía)
Atributos continuos	no	umbral elegido al aprender	umbral en el punto medio
Atributos nominales	sí	sí	codificarlos (one-hot)
Valores faltantes	no	fracciones del ejemplo	NaN, desde la versión 1.3
Poda	no	pospoda pesimista	prepoda y costo-complejidad
Regresión	no	no	sí

Un árbol es inestable

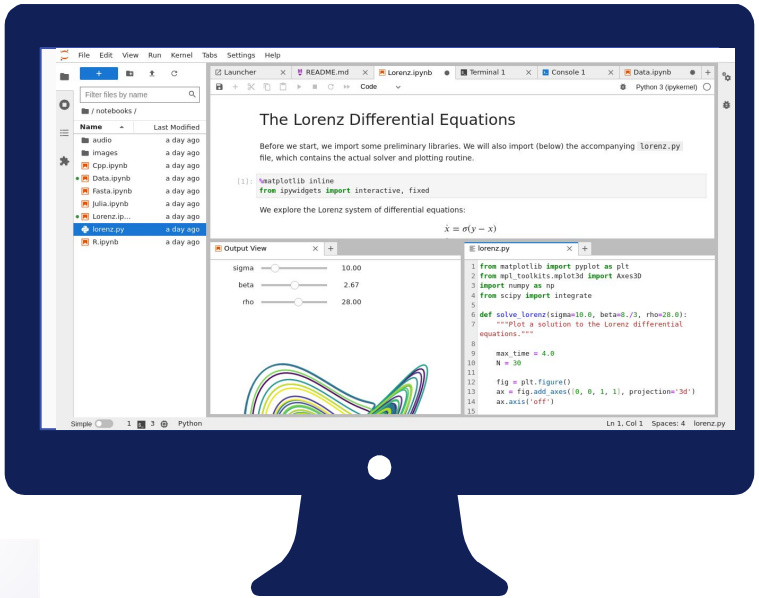
- Un solo ejemplo ruidoso cambió la rama Sunny.
- Con los mismos 14 días, scikit-learn da dos árboles distintos según la semilla: hay cortes empatados.
- En el vino, dos árboles entrenados con mitades distintas clasifican distinto **casi la mitad** de los vinos de prueba. La raíz es la misma, alcohol; lo que cambia está abajo.



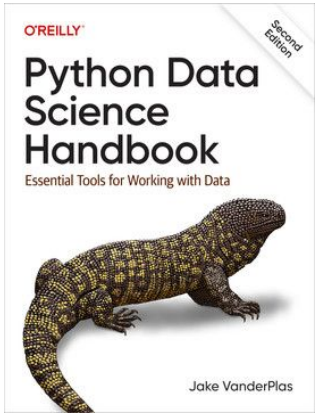
Vinos de prueba en que difieren los dos modelos. Promedio de 20 repeticiones; la línea va del mínimo al máximo.

Promediar muchos árboles reduce esa variación: es la idea de Random Forest (sesión 18).

(Go to live notebook)



Extra Libro



GitHub

- 05.08-Random-Forests.ipynb (primera parte)

Gracias!

¿Alguna pregunta?

hussein@cicese.edu.mx



SCAN ME